Good Pratice API

REST API

BASE DE NOMMAGE

- noms clairs, explicites et cohérents pour les points d'extrémité de l'API, facilitant ainsi la compréhension et l'utilisation par les développeurs.
- Les points d'extrémité doivent représenter un objet à manipuler
- Nommer les ressources en utilisant le pluriel.

  - ~~/v1/usr~~
  - /v1/users
  - /users/123/posts pour accéder aux publications de l'utilisateur 123.

Méthodes HTTP et Actions

- Pour manipuler les objets nous utilisons les verbes HTTP (GET / PUT / POST / DELETE).

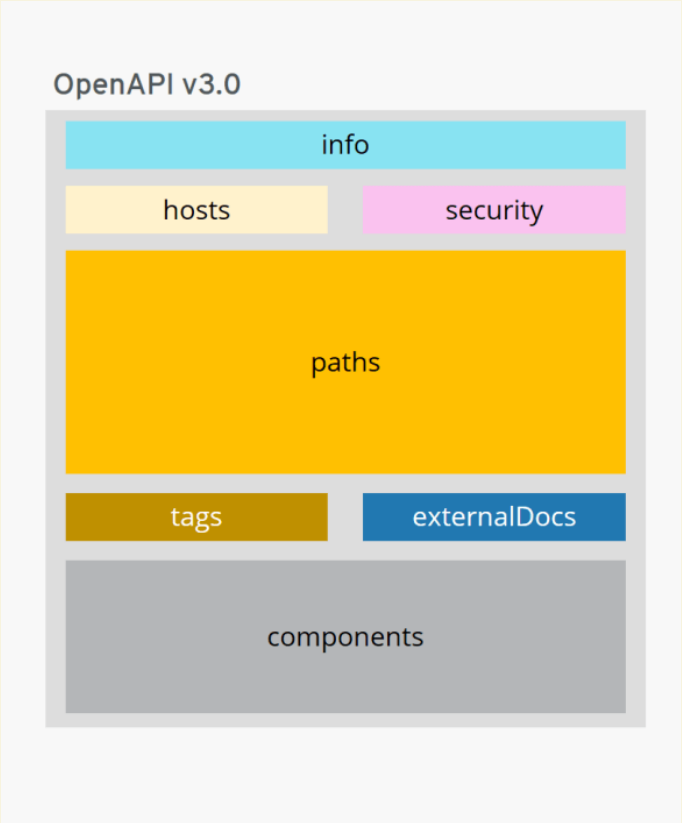
  - ~~/v1/users/create~~
  - POST /v1/users pour créer un utilisateur
  - GET /v1/users pour récupérer un utilisateur

Documentation de l'API

Pour documenter votre application, plusieurs façon existe mais nous allons nous concentrer sur la norme openAPI 3

Structure de base OpenAPI

les sections principales : info, servers, paths, components...



Structure de base OpenAPI

- **Info Object** : version, nom etc. de l'API
- **Contact Object** : informations de contact du fournisseur de l'API
- **License Object** : licence sous laquelle l'API propose ses données
- **Server Object** : nom de l'hôte, structure URL et ports du serveur sur lequel sera interrogée l'API
- **Components Object** : composants encapsulés qui sont utilisés plusieurs fois dans la définition API
- **Paths Object** : chemins d'accès relatifs aux points de terminaison de l'API, utilisés avec le Server Object
- **Path Item Object** : pour les opérations autorisées à prendre un chemin spécifique, comme GET, PUT, POST, DELETE
- **Operation Object** : définit entre autres les paramètres et les réponses du serveur attendues pour une opération

Structure de base OpenAPI

```yaml
info:
  title: Sample API
  description: Optional multiline or single-line description
servers:
  - url: http://api.example.com/v1
    description: Optional server description, e.g. Main (production) server
  - url: http://staging-api.example.com
    description: Optional server description, e.g. Internal staging server for testing
```

Structure de base OpenAPI

```yaml

paths:
  /users:
    get:
      summary: Returns a list of users.
      description: Optional extended description in CommonMark or HTML.
      responses:
        '200':    # status code
          description: A JSON array of user names
          content:
            application/json:
              schema:
                type: array
                items:
                  type: string
```

Structure de base OpenAPI

path avec paramètres

```yaml
paths:
  /users/{userId}:
    get:
      summary: Returns a user by ID.
      parameters:
        - name: userId
          in: path
          required: true
          description: Parameter description in CommonMark or HTML.
          schema:
            type : integer
            format: int64
            minimum: 1
      responses:
        '200':
          description: OK
```

Structure de base OpenAPI

path avec un body (POST)

```yaml
paths:
  /users:
    post:
      summary: Creates a user.
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                username:
                  type: string
      responses:
        '201':
          description: Created
```

Structure de base OpenAPI

path avec une réponse

```yaml
paths:
  /users/{userId}:
    get:
      summary: Returns a user by ID.
      parameters:
        - name: userId
          in: path
          required: true
          description: The ID of the user to return.
          schema:
            type: integer
            format: int64
            minimum: 1
      responses:
        '200':
          description: A user object.
          content:
            application/json:
              schema:
                type: object
                properties:
                  id:
                    type: integer
                    format: int64
                    example: 4
                  name:
                    type: string
                    example: Jessica Smith
        '400':
          description: The specified user ID is invalid (not a number).
        '404':
          description: A user with the specified ID was not found.
        default:
          description: Unexpected error
```

Structure de base OpenAPI

les composants réutilisables :

```yaml
components:
  schemas:
    User:
      type: object
      properties:
        id:
          type: integer
          example: 4
        name:
          type: string
          example: Arthur Dent
      # Both properties are required
      required:
        - id
        - name
```

```yaml
paths:
  /users/{userId}:
    get:
      summary: Returns a user by ID.
      parameters:
        - in: path
          name: userId
          required: true
          schema:
            type: integer
            format: int64
            minimum: 1
      responses:
        '200':
          description: OK
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/User'
  /users:
    post:
      summary: Creates a new user.
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/User'
      responses:
        '201':
          description: Created
```

CODE HTTP

CODE HTTP

- 2XX : succès de la requête
- 3XX : redirection
- 4XX : Erreur du client
- 5XX : Erreur Serveur

CODE HTTP 200

200	OK	RFC 19458	Requête traitée avec succès. La réponse dépendra de la méthode de requête utilisée.
201	Created	RFC 19458	Requête traitée avec succès et création d’un document.
202	Accepted	RFC 19458	Requête traitée, mais sans garantie de résultat.
204	No Content	RFC 19458	Requête traitée avec succès mais pas d’information à renvoyer.

CODE HTTP 400

400	Bad Request	RFC 19458	La syntaxe de la requête est erronée.
401	Unauthorized	RFC 19458	Une authentification est nécessaire pour accéder à la ressource.
403	Forbidden	RFC 19458	Le serveur a compris la requête, mais refuse de l'exécuter. Contrairement à l'erreur 401, s'authentifier ne fera aucune différence. Sur les serveurs où l'authentification est requise, cela signifie généralement que l'authentification a été acceptée mais que les droits d'accès ne permettent pas au client d'accéder à la ressource.
404	Not Found	RFC 19458	Ressource non trouvée.

CODE HTTP 500

500	Internal Server Error	RFC 19458	Erreur interne du serveur.
501	Not Implemented	RFC 19458	Fonctionnalité réclamée non supportée par le serveur.
503	Service Unavailable	RFC 19458	Service temporairement indisponible ou en maintenance.

SECURISER SON API

LES ENTREES

- Ne jamais faire confiance à son utilisateur
- Vérifier que tous les paramètres requis sont présent et au bon format
- Renvoyer une erreur si ce n'est pas le cas
- Utiliser une librairie de validation comme JOI pour s'aider
- Renvoyer une erreur logique a l'utilisateur

JOI

Exemple d'utilisation :

```
1  const Joi = require('joi');
2
3  const schema = Joi.object({
4      username: Joi.string()
5          .alphanum()
6          .min(3)
7          .max(30)
8          .required(),
9
10     password: Joi.string()
11         .pattern(new RegExp('^[a-zA-Z0-9]{3,30}$')),
12
13     access_token: [
14         Joi.string(),
15         Joi.number()
16     ],
17
18     birth_year: Joi.number()
19         .integer()
20         .min(1900)
21         .max(2013)
22 })
```

JOI

Exemple d'utilisation :

```
1
2  schema.validate({ username: 'abc', birth_year: 1994 });
3  // -> { value: { username: 'abc', birth_year: 1994 } }
4
5  schema.validate({});
6  // -> { value: {}, error: '"username" is required' }
7
8  // Also -
9
10 try {
11     const value = await schema.validateAsync({ username: 'abc', birth_year: 1994 });
12 }
13 catch (err) { }
```

LES ENTREES

- Se baser sur le retour de JOI pour renvoyer une erreur explicite à l'utilisateur
- Une erreur de syntaxe = 401

```
1  {
2      "errors": [
3          {
4              "error": "auth-0001",
5              "message": "Incorrect username and password",
6              "detail": "Ensure that the username and password included in the request are correct",
7              "help": "https://example.com/help/error/auth-0001"
8          },
9          ...
10     ]
11 }
```

LES ERREURS SERVEURS

- Ne pas renvoyer la stacktrace à l'utilisateurs
- Essayer d'être le plus descriptif possible

```
1  {
2      "errors": [
3          {
4              "error": "database-0001",
5              "message": "The database is down",
6              "help": "https://example.com/help/error/database-0001"
7          },
8          ...
9      ]
10 }
```

LES SORTIES

- Ne pas renvoyer l'objet de la base de données
- Filtrer les champs que vous ne voulez pas donner à votre utilisateur (par ex: le mot de passe)